

Python Lecture

Data Analysis with Pandas

Data Analysis with Matplotlib

Reading and Writing CSV and Text Files

Working with JSON files

Introduction to Web Scraping and Ethical Consideration

Using requests and BeautifulSoup

Extracting Data from Web pages

Introduction to API's and JSON Format

Making API Request

Making API Request using Request Module

Parsing and Using API Data

Authentication and security basics for APIs in Python

Pandas

- Pandas contains high-level data structures and manipulation tools designed to make data analysis fast and easy in Python.:
 - Data structures with labeled axes supporting automatic or explicit data alignment. This prevents common errors resulting from misaligned data and working with differently-indexed data coming from different sources.
 - Integrated time series functionality.
 - The same data structures handle both time series data and non-time series data.
 - Arithmetic operations and reductions (like summing across an axis) would pass on the metadata (axis labels).
 - Flexible handling of missing data.

Series

4

- A Series is a one-dimensional array-like object containing an array of data (of any NumPy data type) and an associated array of data labels, called its index. The simplest Series is formed from only an array of data:

```
In [1]: from pandas import Series, DataFrame
```

```
In [2]: import pandas as pd
```

```
In [4]: obj = Series([4, 7, -5, 3])
```

```
In [5]: obj
```

```
Out[5]:
```

```
0    4
1    7
2   -5
3    3
```

```
In [6]: obj.values
```

```
Out[6]: array([ 4,  7, -5,  3])
```

```
In [7]: obj.index
```

```
Out[7]: Int64Index([0, 1, 2, 3])
```

```
In [8]: obj2 = Series([4, 7, -5, 3], index=['d', 'b', 'a', 'c'])
```

```
In [9]: obj2
```

```
Out[9]:
```

```
d    4
b    7
a   -5
c    3
```

```
In [11]: obj2['a']
```

```
Out[11]: -5
```

```
In [12]: obj2['d'] = 6
```

Basic Operations on Series

5

```
In [14]: obj2
```

```
Out[14]:
```

```
d    6
b    7
a   -5
c    3
```

```
In [18]: 'b' in obj2
```

```
Out[18]: True
```

```
In [19]: 'e' in obj2
```

```
Out[19]: False
```

```
In [15]: obj2[obj2 > 0]
```

```
Out[15]:
```

```
d    6
b    7
c    3
```

```
In [16]: obj2 * 2
```

```
Out[16]:
```

```
d    12
b    14
a   -10
c     6
```

```
In [17]: np.exp(obj2)
```

```
Out[17]:
```

```
d    403.428793
b   1096.633158
a     0.006738
c    20.085537
```

Series with dictionary

6

```
In [20]: sdata = {'Ohio': 35000, 'Texas': 71000, 'Oregon': 16000, 'Utah': 5000}
```

```
In [21]: obj3 = Series(sdata)
```

```
In [22]: obj3
```

```
Out[22]:
```

Ohio	35000
Oregon	16000
Texas	71000
Utah	5000

```
In [23]: states = ['California', 'Ohio', 'Oregon', 'Texas']
```

```
In [24]: obj4 = Series(sdata, index=states)
```

```
In [25]: obj4
```

```
Out[25]:
```

California	NaN
Ohio	35000
Oregon	16000
Texas	71000

```
In [28]: obj4.isnull()
```

```
Out[28]:
```

California	True
Ohio	False
Oregon	False
Texas	False

DataFrame

7

- A DataFrame represents a tabular, spreadsheet-like data structure containing an ordered collection of columns, each of which can be a different value type (numeric, string, boolean, etc.). The DataFrame has both a row and column index; it can be thought of as a dict of Series (one for all sharing the same index).

```
data = {'state': ['Ohio', 'Ohio', 'Ohio', 'Nevada', 'Nevada'],  
        'year': [2000, 2001, 2002, 2001, 2002],  
        'pop': [1.5, 1.7, 3.6, 2.4, 2.9]}
```

```
frame = DataFrame(data)
```

```
In [38]: frame
```

```
Out[38]:
```

	pop	state	year
0	1.5	Ohio	2000
1	1.7	Ohio	2001
2	3.6	Ohio	2002
3	2.4	Nevada	2001
4	2.9	Nevada	2002

DataFrame

8

```
In [39]: DataFrame(data, columns=['year', 'state', 'pop'])
```

```
Out[39]:
```

	year	state	pop
0	2000	Ohio	1.5
1	2001	Ohio	1.7
2	2002	Ohio	3.6
3	2001	Nevada	2.4
4	2002	Nevada	2.9

```
In [40]: frame2 = DataFrame(data, columns=['year', 'state', 'pop', 'debt'],  
.....:                        index=['one', 'two', 'three', 'four', 'five'])
```

```
In [41]: frame2
```

```
Out[41]:
```

	year	state	pop	debt
one	2000	Ohio	1.5	NaN
two	2001	Ohio	1.7	NaN
three	2002	Ohio	3.6	NaN
four	2001	Nevada	2.4	NaN
five	2002	Nevada	2.9	NaN

```
In [42]: frame2.columns
```

```
Out[42]: Index([year, state, pop, debt], dtype=object)
```

DataFrame

9

- Manipulation in dataframes

```
In [48]: frame2['debt'] = np.arange(5.)
```

```
In [49]: frame2
```

```
Out[49]:
```

	year	state	pop	debt
one	2000	Ohio	1.5	0
two	2001	Ohio	1.7	1
three	2002	Ohio	3.6	2
four	2001	Nevada	2.4	3
five	2002	Nevada	2.9	4

Filling values using arange

Inserting values in **debt** column

```
In [50]: val = Series([-1.2, -1.5, -1.7], index=['two', 'four', 'five'])
```

```
In [51]: frame2['debt'] = val
```

```
In [52]: frame2
```

```
Out[52]:
```

	year	state	pop	debt
one	2000	Ohio	1.5	NaN
two	2001	Ohio	1.7	-1.2
three	2002	Ohio	3.6	NaN
four	2001	Nevada	2.4	-1.5
five	2002	Nevada	2.9	-1.7

DataFrame

10

- Adding eastern column in frame2 and deleting it

```
In [53]: frame2['eastern'] = frame2.state == 'Ohio'
```

```
In [54]: frame2
```

```
Out[54]:
```

	year	state	pop	debt	eastern
one	2000	Ohio	1.5	NaN	True
two	2001	Ohio	1.7	-1.2	True
three	2002	Ohio	3.6	NaN	True
four	2001	Nevada	2.4	-1.5	False
five	2002	Nevada	2.9	-1.7	False

```
In [55]: del frame2['eastern']
```

```
In [56]: frame2.columns
```

```
Out[56]: Index([year, state, pop, debt], dtype=object)
```

DataFrame

11

- Index methods and properties

Method	Description
<code>append</code>	Concatenate with additional Index objects, producing a new Index
<code>diff</code>	Compute set difference as an Index
<code>intersection</code>	Compute set intersection
<code>union</code>	Compute set union
<code>isin</code>	Compute boolean array indicating whether each value is contained in the passed collection
<code>delete</code>	Compute new Index with element at index <code>i</code> deleted
<code>drop</code>	Compute new index by deleting passed values
<code>insert</code>	Compute new Index by inserting element at index <code>i</code>
<code>is_monotonic</code>	Returns True if each element is greater than or equal to the previous element
<code>is_unique</code>	Returns True if the Index has no duplicate values
<code>unique</code>	Compute the array of unique values in the Index

DataFrame

12

- Re-indexing

```
In [79]: obj = Series([4.5, 7.2, -5.3, 3.6], index=['d', 'b', 'a', 'c'])
```

```
In [80]: obj
```

```
Out[80]:
```

```
d    4.5  
b    7.2  
a   -5.3  
c    3.6
```

```
In [83]: obj.reindex(['a', 'b', 'c', 'd', 'e'], fill_value=0)
```

```
Out[83]:
```

```
a   -5.3  
b    7.2  
c    3.6  
d    4.5  
e    0.0
```

DataFrame

13

- Dropping entries from axis

```
In [94]: obj = Series(np.arange(5.), index=['a', 'b', 'c', 'd', 'e'])
```

```
In [95]: new_obj = obj.drop('c')
```

```
In [96]: new_obj
```

```
Out[96]:
```

```
a    0  
b    1  
d    3  
e    4
```

```
In [97]: obj.drop(['d', 'c'])
```

```
Out[97]:
```

```
a    0  
b    1  
e    4
```

DataFrame

14

- Selection and filtering

```
In [112]: data = DataFrame(np.arange(16).reshape((4, 4)),  
.....:                    index=['Ohio', 'Colorado', 'Utah', 'New York'],  
.....:                    columns=['one', 'two', 'three', 'four'])
```

```
In [113]: data
```

```
Out[113]:
```

	one	two	three	four
Ohio	0	1	2	3
Colorado	4	5	6	7
Utah	8	9	10	11
New York	12	13	14	15

```
In [114]: data['two']
```

```
Out[114]:
```

Ohio	1
Colorado	5
Utah	9
New York	13

Name: two

```
In [115]: data[['three', 'one']]
```

```
Out[115]:
```

	three	one
Ohio	2	0
Colorado	6	4
Utah	10	8
New York	14	12

DataFrame

15

- Selection and filtering

```
In [116]: data[:2]
```

```
Out[116]:
```

	one	two	three	four
Ohio	0	1	2	3
Colorado	4	5	6	7

```
In [118]: data < 5
```

```
Out[118]:
```

	one	two	three	four
Ohio	True	True	True	True
Colorado	True	False	False	False
Utah	False	False	False	False
New York	False	False	False	False

```
In [117]: data[data['three'] > 5]
```

```
Out[117]:
```

	one	two	three	four
Colorado	4	5	6	7
Utah	8	9	10	11
New York	12	13	14	15

```
In [119]: data[data < 5] = 0
```

```
In [120]: data
```

```
Out[120]:
```

	one	two	three	four
Ohio	0	0	0	0
Colorado	0	5	6	7
Utah	8	9	10	11
New York	12	13	14	15

DataFrame

16

- Arithmetic and data alignment

```
In [126]: s1 = Series([7.3, -2.5, 3.4, 1.5], index=['a', 'c', 'd', 'e'])
```

```
In [127]: s2 = Series([-2.1, 3.6, -1.5, 4, 3.1], index=['a', 'c', 'e', 'f', 'g'])
```

```
In [128]: s1
```

```
Out[128]:
```

```
a    7.3
c   -2.5
d    3.4
e    1.5
```

```
In [129]: s2
```

```
Out[129]:
```

```
a   -2.1
c    3.6
e   -1.5
f    4.0
g    3.1
```

Adding these together yields:

```
In [130]: s1 + s2
```

```
Out[130]:
```

```
a    5.2
c    1.1
d    NaN
e    0.0
f    NaN
g    NaN
```

DataFrame

17

- Arithmetic and data alignment

```
In [131]: df1 = DataFrame(np.arange(9.).reshape((3, 3)), columns=list('bcd'),
.....:                  index=['Ohio', 'Texas', 'Colorado'])
```

```
In [132]: df2 = DataFrame(np.arange(12.).reshape((4, 3)), columns=list('bde'),
.....:                  index=['Utah', 'Ohio', 'Texas', 'Oregon'])
```

```
In [133]: df1
Out[133]:
```

	b	c	d
Ohio	0	1	2
Texas	3	4	5
Colorado	6	7	8

```
In [134]: df2
Out[134]:
```

	b	d	e
Utah	0	1	2
Ohio	3	4	5
Texas	6	7	8
Oregon	9	10	11

```
In [135]: df1 + df2
Out[135]:
```

	b	c	d	e
Colorado	NaN	NaN	NaN	NaN
Ohio	3	NaN	6	NaN
Oregon	NaN	NaN	NaN	NaN
Texas	9	NaN	12	NaN
Utah	NaN	NaN	NaN	NaN

- Sorting and Ranking

```
In [171]: frame = DataFrame(np.arange(8).reshape((2, 4)), index=['three', 'one'],  
.....:                      columns=['d', 'a', 'b', 'c'])
```

```
In [172]: frame.sort_index()
```

```
Out[172]:
```

	d	a	b	c
one	4	5	6	7
three	0	1	2	3

```
In [173]: frame.sort_index(axis=1)
```

```
Out[173]:
```

	a	b	c	d
three	1	2	3	0
one	5	6	7	4

- Sorting in descending order

```
In [174]: frame.sort_index(axis=1, ascending=False)
```

```
Out[174]:
```

	d	c	b	a
three	0	3	2	1
one	4	7	6	5

DataFrame

19

- Computing Descriptive Statistics

```
In [198]: df = DataFrame([[1.4, np.nan], [7.1, -4.5],  
.....:                  [np.nan, np.nan], [0.75, -1.3]],  
.....:                  index=['a', 'b', 'c', 'd'],  
.....:                  columns=['one', 'two'])
```

```
In [199]: df
```

```
Out[199]:
```

	one	two
a	1.40	NaN
b	7.10	-4.5
c	NaN	NaN
d	0.75	-1.3

```
In [200]: df.sum()
```

```
Out[200]:
```

one	9.25
two	-5.80

Passing axis=1 sums over the rows instead:

```
In [201]: df.sum(axis=1)
```

```
Out[201]:
```

a	1.40
b	2.60
c	NaN
d	-0.55

DataFrame

20

- Computing Descriptive Statistics

```
In [202]: df.mean(axis=1, skipna=False)
```

```
Out[202]:
```

```
a      NaN
b      1.300
c      NaN
d     -0.275
```

Method	Description
--------	-------------

axis	Axis to reduce over. 0 for DataFrame's rows and 1 for columns.
------	--

skipna	Exclude missing values, True by default.
--------	--

- Some methods, like `idxmin` and `idxmax`, return indirect statistics like the index value where the minimum or maximum values are attained:

```
In [203]: df.idxmax()
```

```
Out[203]:
```

```
one      b
two      d
```

DataFrame

21

- Computing Descriptive Statistics
 - Describe is function which producing multiple summary statistics in one shot

```
In [205]: df.describe()
```

```
Out[205]:
```

	one	two
count	3.000000	2.000000
mean	3.083333	-2.900000
std	3.493685	2.262742
min	0.750000	-4.500000
25%	1.075000	-3.700000
50%	1.400000	-2.900000
75%	4.250000	-2.100000
max	7.100000	-1.300000

DataFrame

22

- Descriptive and summary statistics

Method	Description
<code>count</code>	Number of non-NA values
<code>describe</code>	Compute set of summary statistics for Series or each DataFrame column
<code>min, max</code>	Compute minimum and maximum values
<code>argmin, argmax</code>	Compute index locations (integers) at which minimum or maximum value obtained, respectively
<code>idxmin, idxmax</code>	Compute index values at which minimum or maximum value obtained, respectively
<code>quantile</code>	Compute sample quantile ranging from 0 to 1
<code>sum</code>	Sum of values
<code>mean</code>	Mean of values
<code>median</code>	Arithmetic median (50% quantile) of values
<code>mad</code>	Mean absolute deviation from mean value
<code>var</code>	Sample variance of values
<code>std</code>	Sample standard deviation of values

- Unique Values, Value Counts, and Membership

```
In [217]: obj = Series(['c', 'a', 'd', 'a', 'a', 'b', 'b', 'c', 'c'])
```

```
In [218]: uniques = obj.unique()
```

```
In [219]: uniques  
Out[219]: array([c, a, d, b], dtype=object)
```

```
In [220]: obj.value_counts()  
Out[220]:
```

c	3
a	3
b	2
d	1

```
In [222]: mask = obj.isin(['b', 'c'])
```

```
In [223]: mask  
Out[223]:
```

0	True
1	False
2	False
3	False
4	False
5	True
6	True
7	True
8	True

```
In [224]: obj[mask]  
Out[224]:
```

0	c
5	b
6	b
7	c
8	c

- Unique Values, Value Counts, and Membership

Method	Description
<code>isin</code>	Compute boolean array indicating whether each Series value is contained in the passed sequence of values.
<code>unique</code>	Compute array of unique values in a Series, returned in the order observed.
<code>value_counts</code>	Return a Series containing unique values as its index and frequencies as its values, ordered count in descending order.

- Handling Missing Data

```
In [229]: string_data = Series(['aardvark', 'artichoke', np.nan, 'avocado'])
```

```
In [230]: string_data
```

```
Out[230]:
```

```
0    aardvark
1    artichoke
2         NaN
3     avocado
```

```
In [231]: string_data.isnull()
```

```
Out[231]:
```

```
0    False
1    False
2     True
3    False
```

Argument	Description
dropna	Filter axis labels based on whether values for each label have missing data, with varying thresholds for how much missing data to tolerate.
fillna	Fill in missing data with some value or using an interpolation method such as 'ffill' or 'bfill'.
isnull	Return like-type object containing boolean values indicating which values are missing / NA.
notnull	Negation of isnull.

DataFrame

26

- Filtering out Missing Data

```
In [234]: from numpy import nan as NA
```

```
In [235]: data = Series([1, NA, 3.5, NA, 7])
```

```
In [236]: data.dropna()
```

```
Out[236]:
```

```
0    1.0
2    3.5
4    7.0
```

```
In [238]: data = DataFrame([[1., 6.5, 3.], [1., NA, NA],
.....:                    [NA, NA, NA], [NA, 6.5, 3.]])
```

```
In [239]: cleaned = data.dropna()
```

```
In [240]: data
```

```
Out[240]:
```

```
      0    1    2
0  1  6.5    3
1  1  NaN  NaN
2 NaN  NaN  NaN
3 NaN  6.5    3
```

```
In [241]: cleaned
```

```
Out[241]:
```

```
      0    1    2
0  1  6.5    3
```

DataFrame

27

- Filtering out Missing Data

- Passing how='all' will only drop rows that are all NA:

```
In [242]: data.dropna(how='all')
```

```
Out[242]:
```

	0	1	2
0	1	6.5	3
1	1	NaN	NaN
3	NaN	6.5	3

```
In [243]: data[4] = NA
```

```
In [244]: data
```

```
Out[244]:
```

	0	1	2	4
0	1	6.5	3	NaN
1	1	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	6.5	3	NaN

```
In [245]: data.dropna(axis=1, how='all')
```

```
Out[245]:
```

	0	1	2
0	1	6.5	3
1	1	NaN	NaN
2	NaN	NaN	NaN
3	NaN	6.5	3

DataFrame

28

- Filling in Missing Data

```
In [250]: df.fillna(0)
```

```
Out[250]:
```

	0	1	2
0	-0.577087	0.000000	0.000000
1	0.523772	0.000000	0.000000
2	-0.713544	0.000000	0.000000
3	-1.860761	0.000000	0.560145
4	-1.265934	0.000000	-1.063512
5	0.332883	-2.359419	-0.199543
6	-1.541996	-0.970736	-1.307030

```
In [251]: df.fillna({1: 0.5, 3: -1})
```

```
Out[251]:
```

	0	1	2
0	-0.577087	0.500000	NaN
1	0.523772	0.500000	NaN
2	-0.713544	0.500000	NaN
3	-1.860761	0.500000	0.560145
4	-1.265934	0.500000	-1.063512
5	0.332883	-2.359419	-0.199543
6	-1.541996	-0.970736	-1.307030

```
In [259]: data = Series([1., NA, 3.5, NA, 7])
```

```
In [260]: data.fillna(data.mean())
```

```
Out[260]:
```

0	1.000000
1	3.833333
2	3.500000
3	3.833333
4	7.000000

- **Hierarchical indexing:**

- Hierarchical indexing is an important feature of pandas enabling you to have multiple (two or more) index levels on an axis.

```
In [261]: data = Series(np.random.randn(10),  
.....:                  index=[['a', 'a', 'a', 'b', 'b', 'b', 'c', 'c', 'd', 'd'],  
.....:                  [1, 2, 3, 1, 2, 3, 1, 2, 2, 3]))
```

```
In [262]: data  
Out[262]:  
a  1    0.670216  
   2    0.852965  
   3   -0.955869  
b  1   -0.023493  
   2   -2.304234  
   3   -0.652469  
c  1   -1.218302  
   2   -1.332610  
d  2    1.074623  
   3    0.723642
```

```
In [263]: data.index  
Out[263]:  
MultiIndex  
[('a', 1) ('a', 2) ('a', 3) ('b', 1) ('b', 2) ('b', 3) ('c', 1)  
 ('c', 2) ('d', 2) ('d', 3)]
```

```
In [265]: data['b':'c']  
Out[265]:  
b  1   -0.023493  
   2   -2.304234  
   3   -0.652469  
c  1   -1.218302  
   2   -1.332610
```

DataFrame

30

- Hierarchical indexing

```
In [270]: frame = DataFrame(np.arange(12).reshape((4, 3)),
.....:                      index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],
.....:                      columns=[['Ohio', 'Ohio', 'Colorado'],
.....:                               ['Green', 'Red', 'Green']])
```

```
In [271]: frame
```

```
Out[271]:
```

		Ohio		Colorado
		Green	Red	Green
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

```
In [272]: frame.index.names = ['key1', 'key2']
```

```
In [273]: frame.columns.names = ['state', 'color']
```

```
In [274]: frame
```

```
Out[274]:
```

		state		Ohio		Colorado
		color		Green	Red	Green
		key1	key2			
a	1			0	1	2
	2			3	4	5
b	1			6	7	8
	2			9	10	11

Data loading, Storage and file format

Data loading, Storage and file format

32

- Parsing function in pandas

Function	Description
<code>read_csv</code>	Load delimited data from a file, URL, or file-like object. Use comma as default delimiter
<code>read_table</code>	Load delimited data from a file, URL, or file-like object. Use tab (' \t ') as default delimiter
<code>read_fwf</code>	Read data in fixed-width column format (that is, no delimiters)

Data loading, Storage and file format

33

- Reading data from csv file

```
import pandas as pd
```

```
In [847]: df = pd.read_csv('ch06/ex1.csv')
```

```
In [848]: df
```

```
Out[848]:
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

```
In [849]: pd.read_table('ch06/ex1.csv', sep=',')
```

```
Out[849]:
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Data loading, Storage and file format

34

- Reading data from csv file without header field

```
In [851]: pd.read_csv('ch06/ex2.csv', header=None)
```

```
Out[851]:
```

	X.1	X.2	X.3	X.4	X.5
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

```
In [852]: pd.read_csv('ch06/ex2.csv', names=['a', 'b', 'c', 'd', 'message'])
```

```
Out[852]:
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Data loading, Storage and file format

35

- Saving a sample csv file without header field

```
In [853]: names = ['a', 'b', 'c', 'd', 'message']
```

```
In [854]: pd.read_csv('ch06/ex2.csv', names=names, index_col='message')
```

```
Out[854]:
```

	a	b	c	d
message				
hello	1	2	3	4
world	5	6	7	8
foo	9	10	11	12

- Skip rows from the file when loading

```
In [862]: pd.read_csv('ch06/ex4.csv', skiprows=[0, 2, 3])
```

```
Out[862]:
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Data loading, Storage and file format

36

- Working with hierarchical indexing

```
In [856]: parsed = pd.read_csv('ch06/csv_mindex.csv', index_col=['key1', 'key2'])
```

```
In [857]: parsed
```

```
Out[857]:
```

		value1	value2
key1	key2		
one	a	1	2
	b	3	4
	c	5	6
	d	7	8
two	a	9	10
	b	11	12
	c	13	14
	d	15	16

Data loading, Storage and file format

37

- Considering a text file

```
In [858]: list(open('ch06/ex3.txt'))
```

```
Out[858]:
```

```
['          A          B          C\n',  
 'aaa -0.264438 -1.026059 -0.619500\n',  
 'bbb  0.927272  0.302904 -0.032399\n',  
 'ccc -0.264273 -0.386314 -0.217601\n',  
 'ddd -0.871858 -0.348382  1.100491\n']
```

```
In [859]: result = pd.read_table('ch06/ex3.txt', sep='\s+')
```

```
In [860]: result
```

```
Out[860]:
```

	A	B	C
aaa	-0.264438	-1.026059	-0.619500
bbb	0.927272	0.302904	-0.032399
ccc	-0.264273	-0.386314	-0.217601
ddd	-0.871858	-0.348382	1.100491

Data loading, Storage and file format

38

- Read_csv/read_table function arguments

Argument	Description
path	String indicating filesystem location, URL, or file-like object
sep or delimiter	Character sequence or regular expression to use to split fields in each row
header	Row number to use as column names. Defaults to 0 (first row), but should be None if there is no header row
index_col	Column numbers or names to use as the row index in the result. Can be a single name/number or a list of them for a hierarchical index
names	List of column names for result, combine with header=None
skiprows	Number of rows at beginning of file to ignore or list of row numbers (starting from 0) to skip
na_values	Sequence of values to replace with NA

Data loading, Storage and file format

39

- Reading text file in pieces

```
In [873]: pd.read_csv('ch06/ex6.csv', nrows=5)
Out[873]:
```

	one	two	three	four	key
0	0.467976	-0.038649	-0.295344	-1.824726	L
1	-0.358893	1.404453	0.704965	-0.200638	B
2	-0.501840	0.659254	-0.421691	-0.057688	G
3	0.204886	1.074134	1.388361	-0.982404	R
4	0.354628	-0.133116	0.283763	-0.837063	Q

- Reading using chunksize as number of rows

```
In [874]: chunker = pd.read_csv('ch06/ex6.csv', chunksize=1000)
```

```
In [875]: chunker
```

```
Out[875]: <pandas.io.parsers.TextParser at 0x8398150>
```

```
chunker = pd.read_csv('ch06/ex6.csv', chunksize=1000)
tot = Series([])
for piece in chunker:
    tot = tot.add(piece['key'].value_counts(), fill_value=0)
```

```
tot[:10]
```

0	151.0
1	146.0
2	152.0
3	162.0
4	171.0
5	157.0
6	166.0
7	164.0
8	162.0
9	150.0

dtype: float64

Data loading, Storage and file format

40

- Reading JSON file

```
In [10]: import json
```

```
In [17]: df = pd.read_json('ch06/ex9.json')  
df
```

Out[17]:

employee	
married	True
name	sonoo
salary	56000

Data loading, Storage and file format

41

- Writing data out to csv

```
In [878]: data = pd.read_csv('ch06/ex5.csv')
```

```
In [879]: data
```

```
Out[879]:
```

	something	a	b	c	d	message
0	one	1	2	3	4	NaN
1	two	5	6	NaN	8	world

```
In [880]: data.to_csv('ch06/out.csv')
```

Data loading, Storage and file format

42

- to csv method with series

```
import numpy as np
```

```
In [886]: dates = pd.date_range('1/1/2000', periods=7)
```

```
In [887]: ts = Series(np.arange(7), index=dates)
```

```
In [888]: ts.to_csv('ch06/tseries.csv')
```

```
df = pd.read_csv('ch06/tseries.csv', parse_dates=True)  
df
```

	Unnamed: 0	0
0	2000-01-01	0
1	2000-01-02	1
2	2000-01-03	2
3	2000-01-04	3
4	2000-01-05	4
5	2000-01-06	5
6	2000-01-07	6

Plotting and Visualization

Matplotlib

- Matplotlib is a (primarily 2D) desktop plotting package designed for creating publication-quality plots.
- It supports many different GUI backends on all operating systems and additionally can export graphics to all of the common vector and raster graphics formats: PDF, SVG, JPG, PNG, BMP, GIF, etc.
- Matplotlib API functions like plot and close are all in the matplotlib.pyplot module, which is typically imported by convention as:

```
import matplotlib.pyplot as plt
```

Matplotlib (Figure and subplots)

```
plt.plot(randn(30).cumsum(), color='k', linestyle='dashed', marker='o')
```

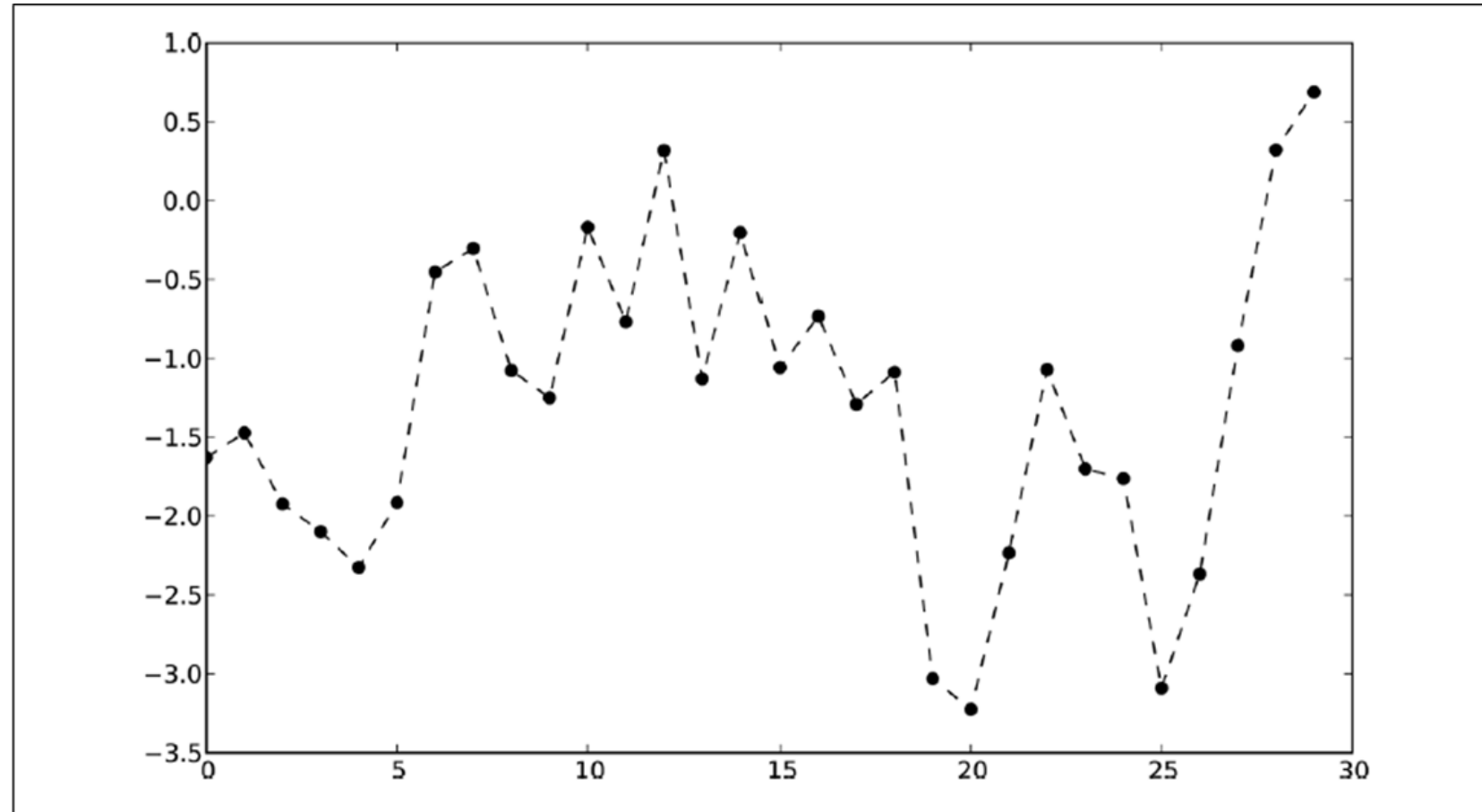


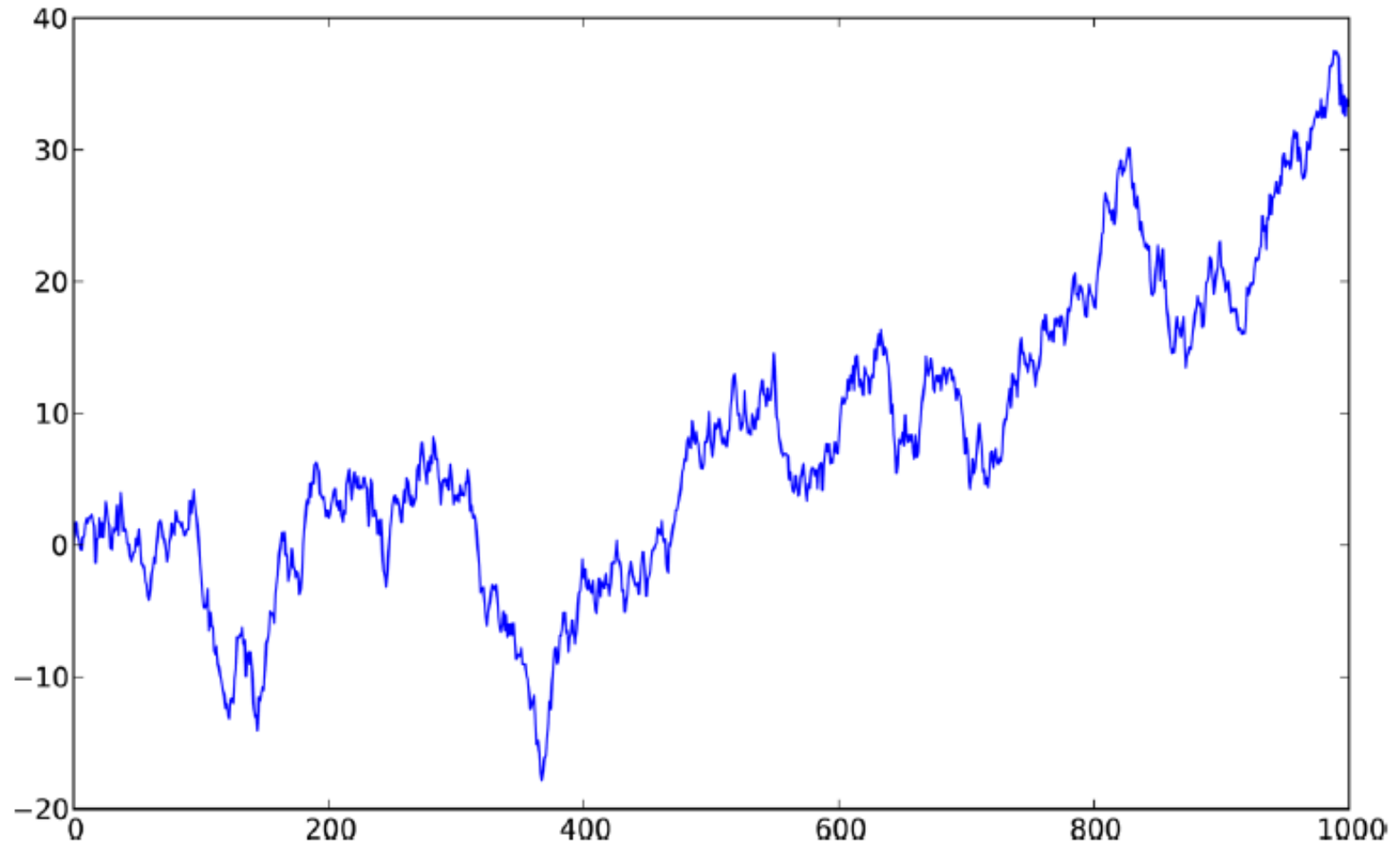
Figure 8-6. Line plot with markers example

Matplotlib (Figure and subplots)

- Setting the title, axis labels, ticks, and ticklabels

```
In [34]: fig = plt.figure(); ax = fig.add_subplot(1, 1, 1)
```

```
In [35]: ax.plot(randn(1000).cumsum())
```



Matplotlib (Figure and subplots)

- **Setting the title, axis labels, ticks, and ticklabels**

Add these command in the previous code

```
In [36]: ticks = ax.set_xticks([0, 250, 500, 750, 1000])
```

```
In [37]: labels = ax.set_xticklabels(['one', 'two', 'three', 'four', 'five'],  
.....:                             rotation=30, fontsize='small')
```

```
In [38]: ax.set_title('My first matplotlib plot')
```

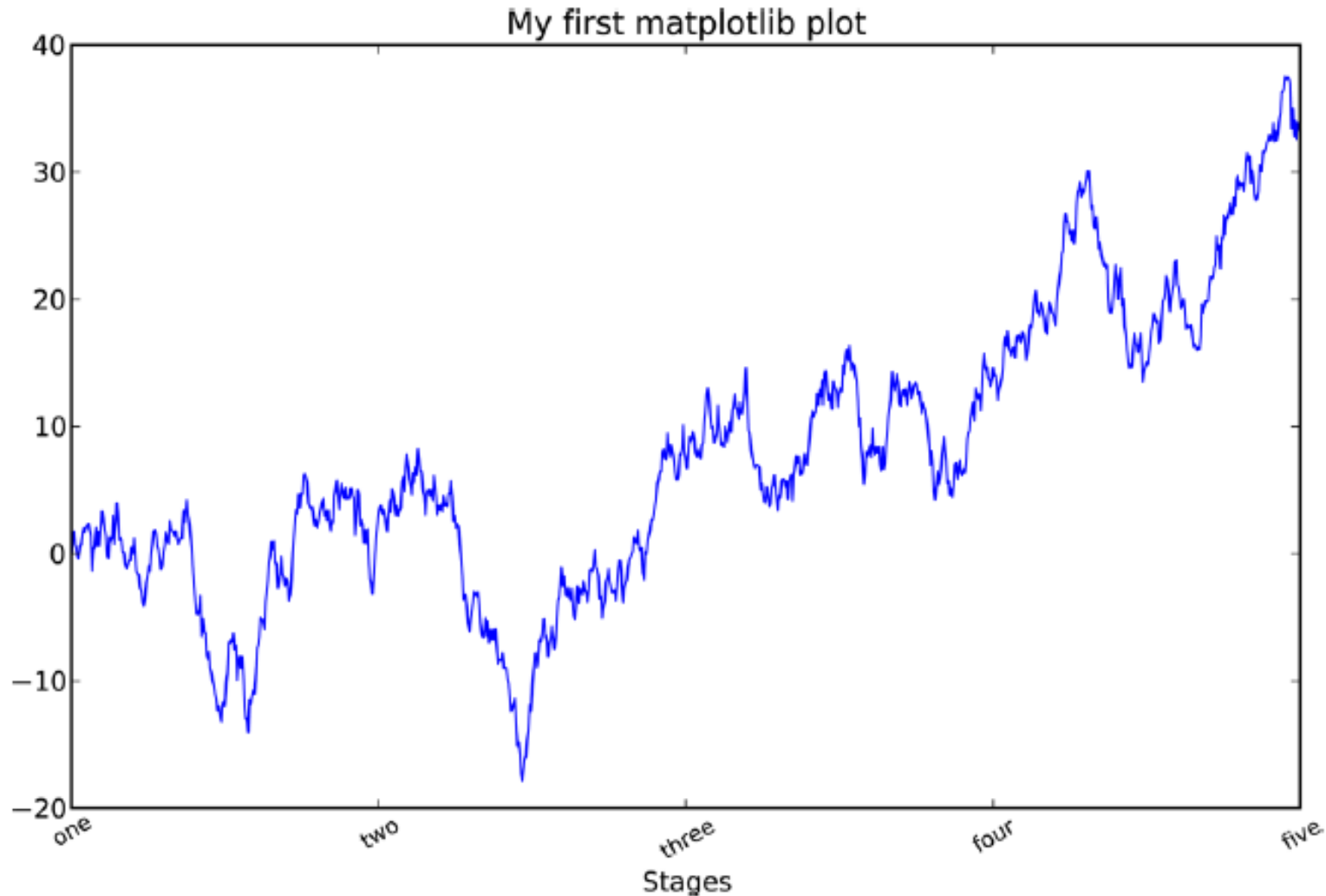
```
Out[38]: <matplotlib.text.Text at 0x7f9190912850>
```

```
In [39]: ax.set_xlabel('Stages')
```

Matplotlib (Figure and subplots)

48

- Setting the title, axis labels, ticks, and ticklabels



Matplotlib (Figure and subplots)

- **Adding legends**

```
In [40]: fig = plt.figure(); ax = fig.add_subplot(1, 1, 1)
```

```
In [41]: ax.plot(randn(1000).cumsum(), 'k', label='one')  
Out[41]: [<matplotlib.lines.Line2D at 0x4720a90>]
```

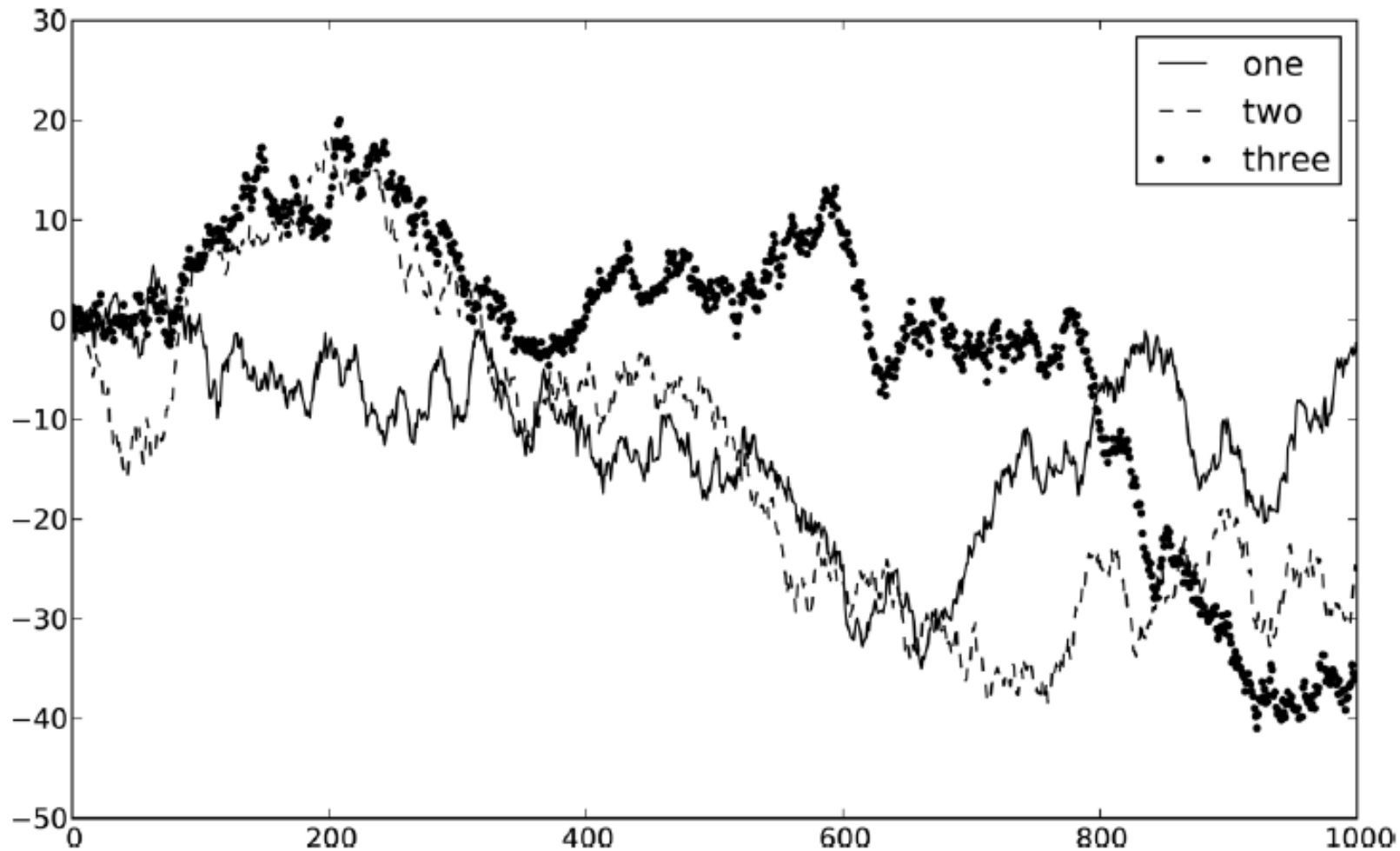
```
In [42]: ax.plot(randn(1000).cumsum(), 'k--', label='two')  
Out[42]: [<matplotlib.lines.Line2D at 0x4720f90>]
```

```
In [43]: ax.plot(randn(1000).cumsum(), 'k.', label='three')  
Out[43]: [<matplotlib.lines.Line2D at 0x4723550>]
```

Matplotlib (Figure and subplots)

50

- Adding legends



Matplotlib (Plotting functions in pandas)

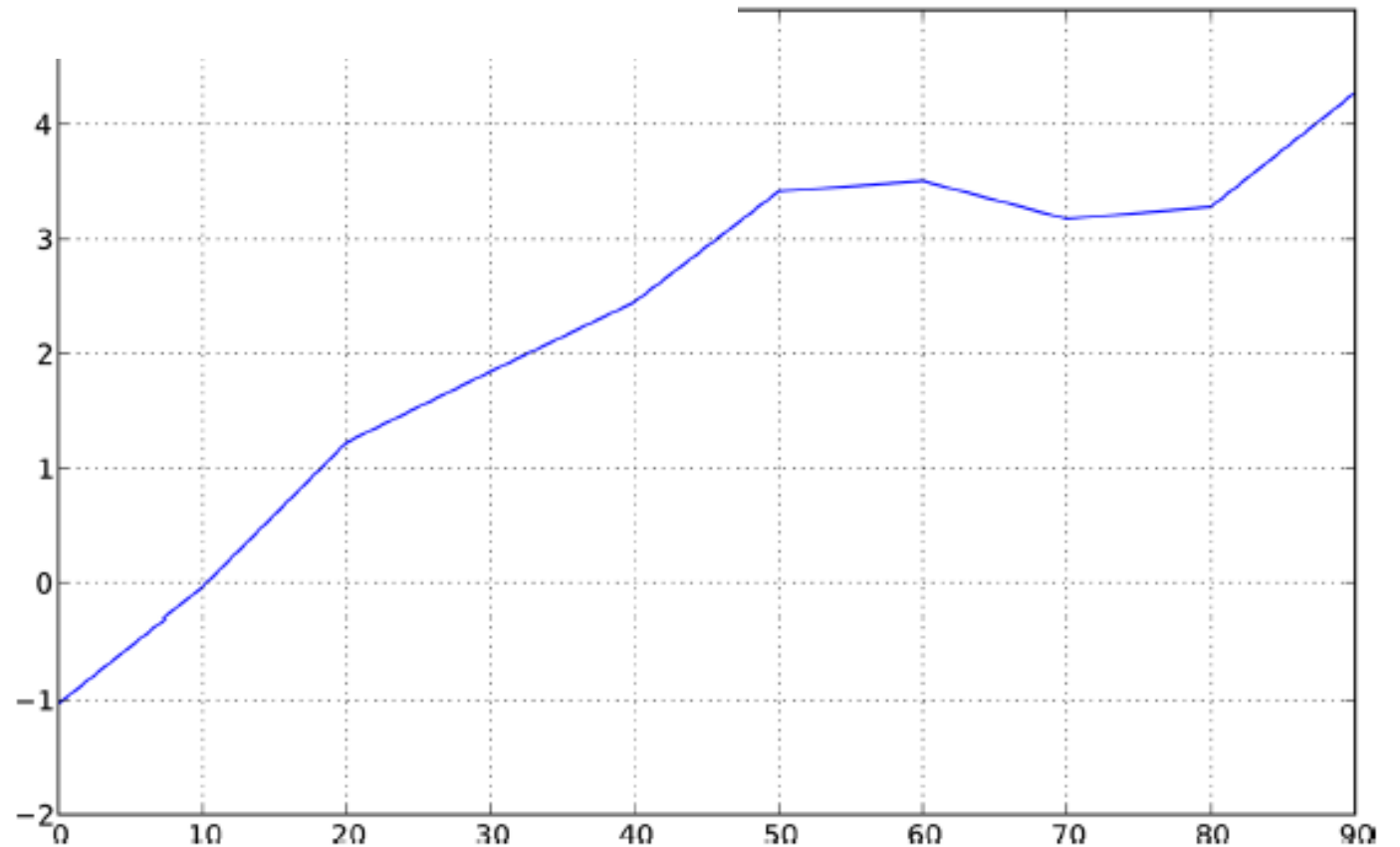
51

- Line Plot

```
from pandas import Series
import pandas as pd
```

```
In [55]: s = Series(np.random.randn(10).cumsum(), index=np.arange(0, 100, 10))
```

```
In [56]: s.plot()
```



Matplotlib (Plotting functions in pandas)

- DataFrame's plot method plots each of its columns as a different line on the same subplot, creating a legend automatically...

```
from pandas import DataFrame
```

```
In [57]: df = DataFrame(np.random.randn(10, 4).cumsum(0),  
.....:                  columns=['A', 'B', 'C', 'D'],  
.....:                  index=np.arange(0, 100, 10))
```

```
In [58]: df.plot()
```

Table 8-3. Series.plot method arguments

Argument	Description
label	Label for plot legend
ax	matplotlib subplot object to plot on. If nothing passed, uses active matplotlib subplot
style	Style string, like 'ko--', to be passed to matplotlib.
alpha	The plot fill opacity (from 0 to 1)

Matplotlib (Plotting functions in pandas)

53

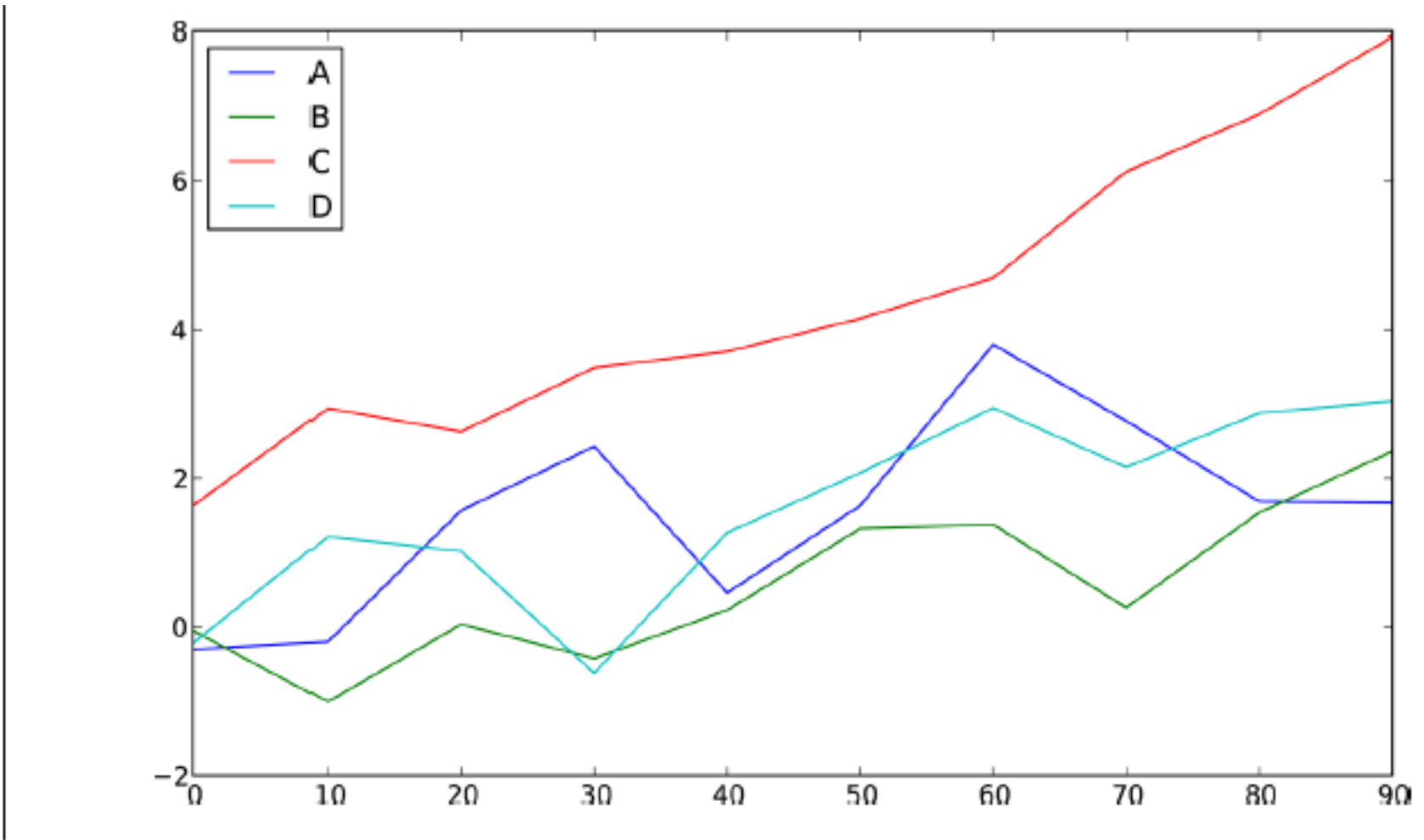


Figure 8-14. Simple DataFrame plot example

Matplotlib (Plotting functions in pandas)

54

Argument	Description
kind	Can be 'line', 'bar', 'barh', 'kde'
logy	Use logarithmic scaling on the Y axis
use_index	Use the object index for tick labels
rot	Rotation of tick labels (0 through 360)
xticks	Values to use for X axis ticks
yticks	Values to use for Y axis ticks
xlim	X axis limits (e.g. [0, 10])
ylim	Y axis limits
grid	Display axis grid (on by default)

Argument	Description
subplots	Plot each DataFrame column in a separate subplot
sharex	If subplots=True, share the same X axis, linking ticks and limits
sharey	If subplots=True, share the same Y axis
figsize	Size of figure to create as tuple
title	Plot title as string
legend	Add a subplot legend (True by default)
sort_columns	Plot columns in alphabetical order; by default uses existing column order

Matplotlib (Plotting functions in pandas)

- **Bar Plots**

With a DataFrame, bar plots group the values in each row together in a group in bars, side by side, for each value.

```
In [63]: df = DataFrame(np.random.rand(6, 4),  
.....:                 index=['one', 'two', 'three', 'four', 'five', 'six'],  
.....:                 columns=pd.Index(['A', 'B', 'C', 'D'], name='Genus'))
```

```
In [64]: df
```

```
Out[64]:
```

Genus	A	B	C	D
one	0.301686	0.156333	0.371943	0.270731
two	0.750589	0.525587	0.689429	0.358974
three	0.381504	0.667707	0.473772	0.632528
four	0.942408	0.180186	0.708284	0.641783
five	0.840278	0.909589	0.010041	0.653207
six	0.062854	0.589813	0.811318	0.060217

```
In [65]: df.plot(kind='bar')
```

Matplotlib (Plotting functions in pandas)

56

- **Bar Plots**

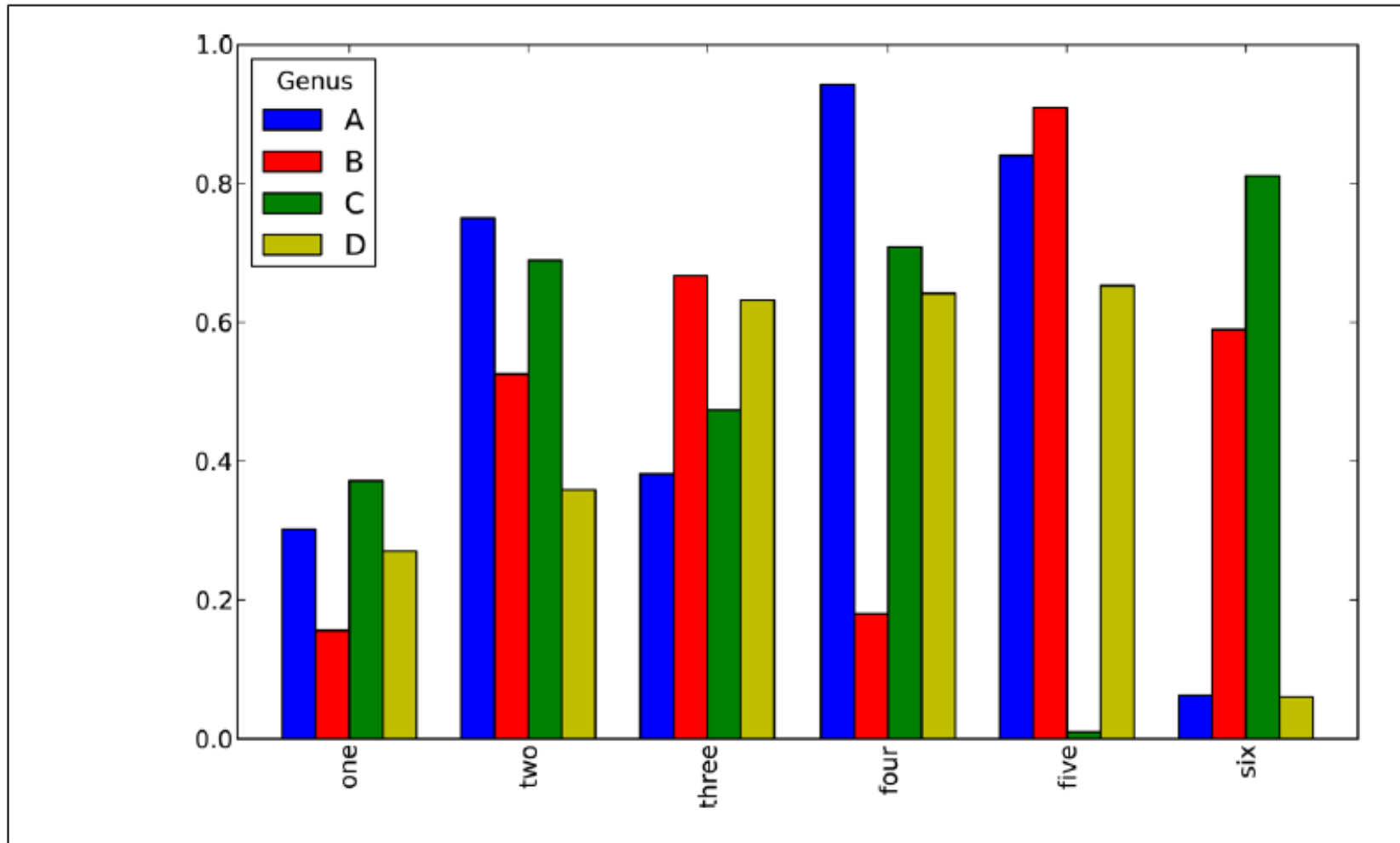


Figure 8-16. DataFrame bar plot example

Matplotlib (Plotting functions in pandas)

- **Bar Plots (Example)**

Suppose we wanted to make a stacked bar plot showing the percentage of data points for each party size on each day. I load the data using `read_csv` and make a cross-tabulation by day and party size

```
In [68]: tips = pd.read_csv('ch08/tips.csv')
```

```
In [69]: party_counts = pd.crosstab(tips.day, tips.size)
```

```
In [70]: party_counts
```

```
Out[70]:
```

size	1	2	3	4	5	6
day						
Fri	1	16	1	1	0	0
Sat	2	53	18	13	1	0
Sun	0	39	15	18	3	1
Thur	1	48	4	5	1	3

Matplotlib (Plotting functions in pandas)

58

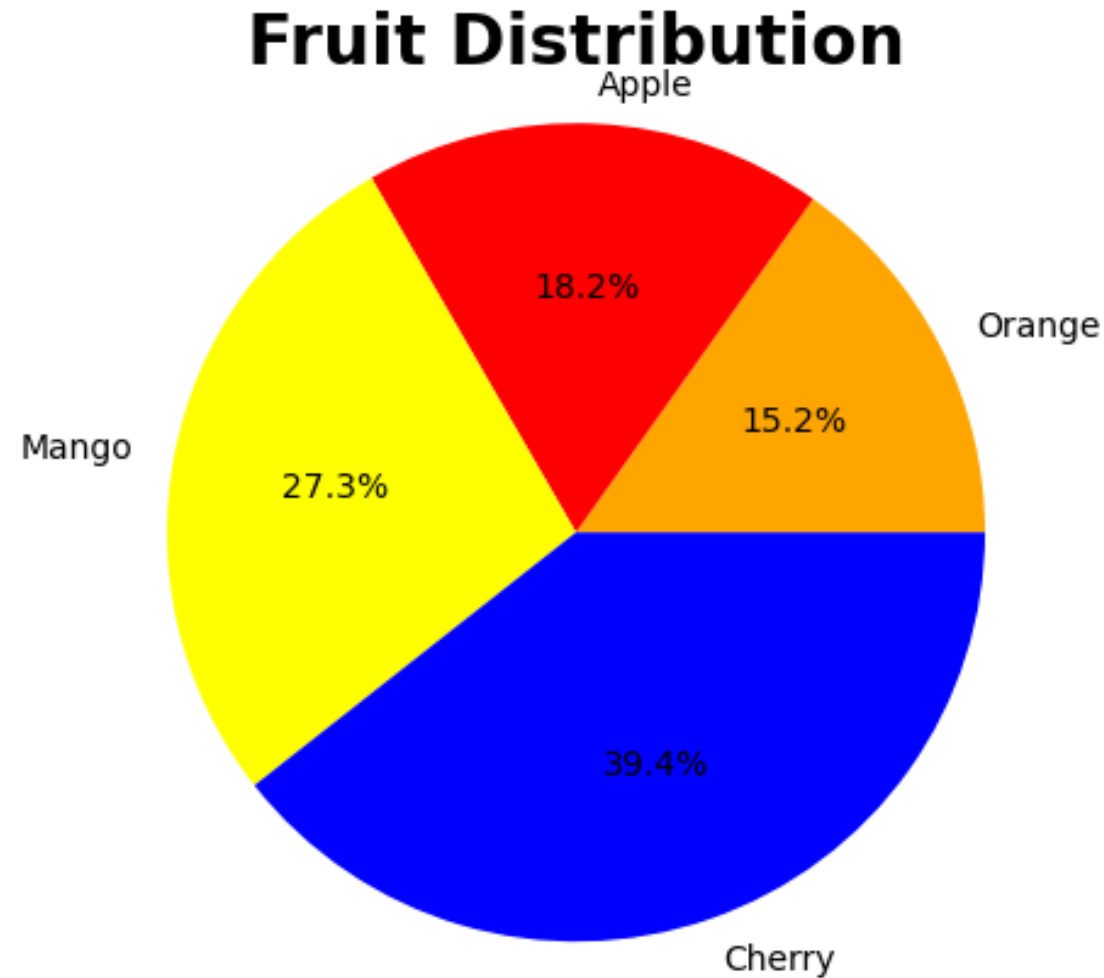
- Pie Chart

```
labels = ['Orange', 'Apple', 'Mango', 'Cherry']
sizes = [25, 30, 45, 65]
colors = ['Orange', 'Red', 'Yellow', 'Blue']
plt.pie(sizes, labels= labels, colors = colors, autopct = '%1.1f%%')
plt.title('Fruit Distribution', fontsize = 20, weight='bold')
plt.axis('equal')
plt.show()
```

Matplotlib (Plotting functions in pandas)

59

- Pie Chart



Matplotlib (Plotting functions in pandas)

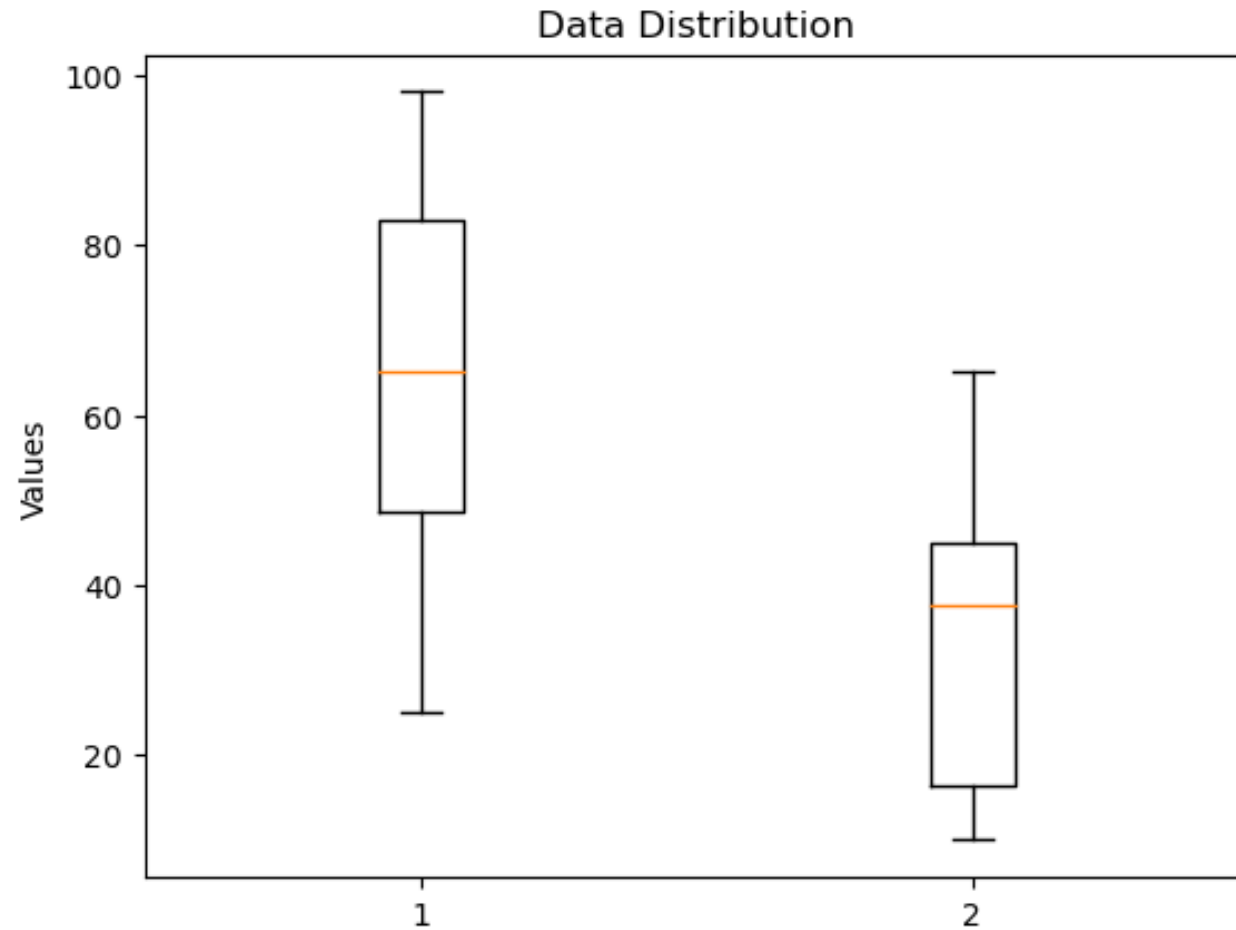
- **Box Plot**

```
data1 = [50,65,78,98,25,45,65,98]
data2 = [45,10,12,65,45,30]
plt.boxplot([data1, data2])
plt.ylabel('Values')
plt.title('Data Distribution')
plt.show()
```

Matplotlib (Plotting functions in pandas)

61

- **Box Plot**



Matplotlib (Plotting functions in pandas)

- **Histograms**

A histogram, with which you may be well-acquainted, is a kind of bar plot that gives a discretized display of value frequency. The data points are split into discrete, evenly spaced bins, and the number of data points in each bin is plotted. Using the tipping data from before, we can make a histogram of tip percentages of the total bill using the hist method on the Series.

```
In [76]: tips['tip_pct'] = tips['tip'] / tips['total_bill']
```

```
In [77]: tips['tip_pct'].hist(bins=50)
```

Matplotlib (Plotting functions in pandas)

63

- **Histograms**

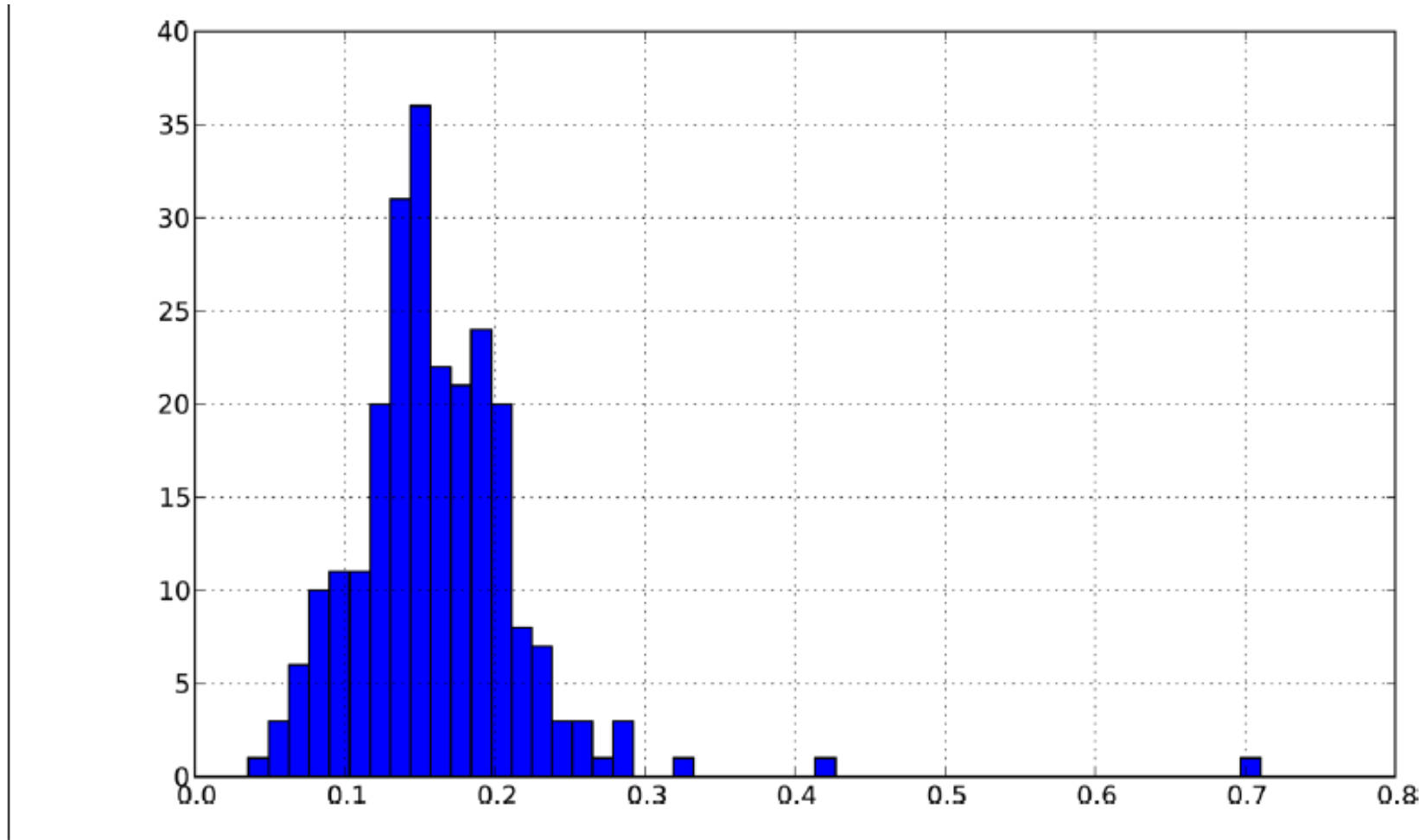


Figure 8-19. Histogram of tip percentages

Matplotlib (Plotting functions in pandas)

■ Scatter Plots

Scatter plots are a useful way of examining the relationship between two one-dimensional data series. matplotlib has a scatter plotting method that is the workhorse of making these kinds of plots. To give an example, I load the macrodata dataset from the statsmodels project, select a few variables, then compute log differences:

```
In [86]: macro = pd.read_csv('ch08/macrodata.csv')
```

```
In [87]: data = macro[['cpi', 'm1', 'tbilrate', 'unemp']]
```

```
In [88]: trans_data = np.log(data).diff().dropna()
```

```
In [89]: trans_data[-5:]
```

```
Out[89]:
```

	cpi	m1	tbilrate	unemp
198	-0.007904	0.045361	-0.396881	0.105361
199	-0.021979	0.066753	-2.277267	0.139762
200	0.002340	0.010286	0.606136	0.160343
201	0.008419	0.037461	-0.200671	0.127339
202	0.008894	0.012202	-0.405465	0.042560

```
plt.scatter(trans_data['m1'], trans_data['unemp'])
plt.title('Changes in log %s vs. log %s' % ('m1', 'unemp'))
```

Matplotlib (Plotting functions in pandas)

65

- **Scatter Plots**

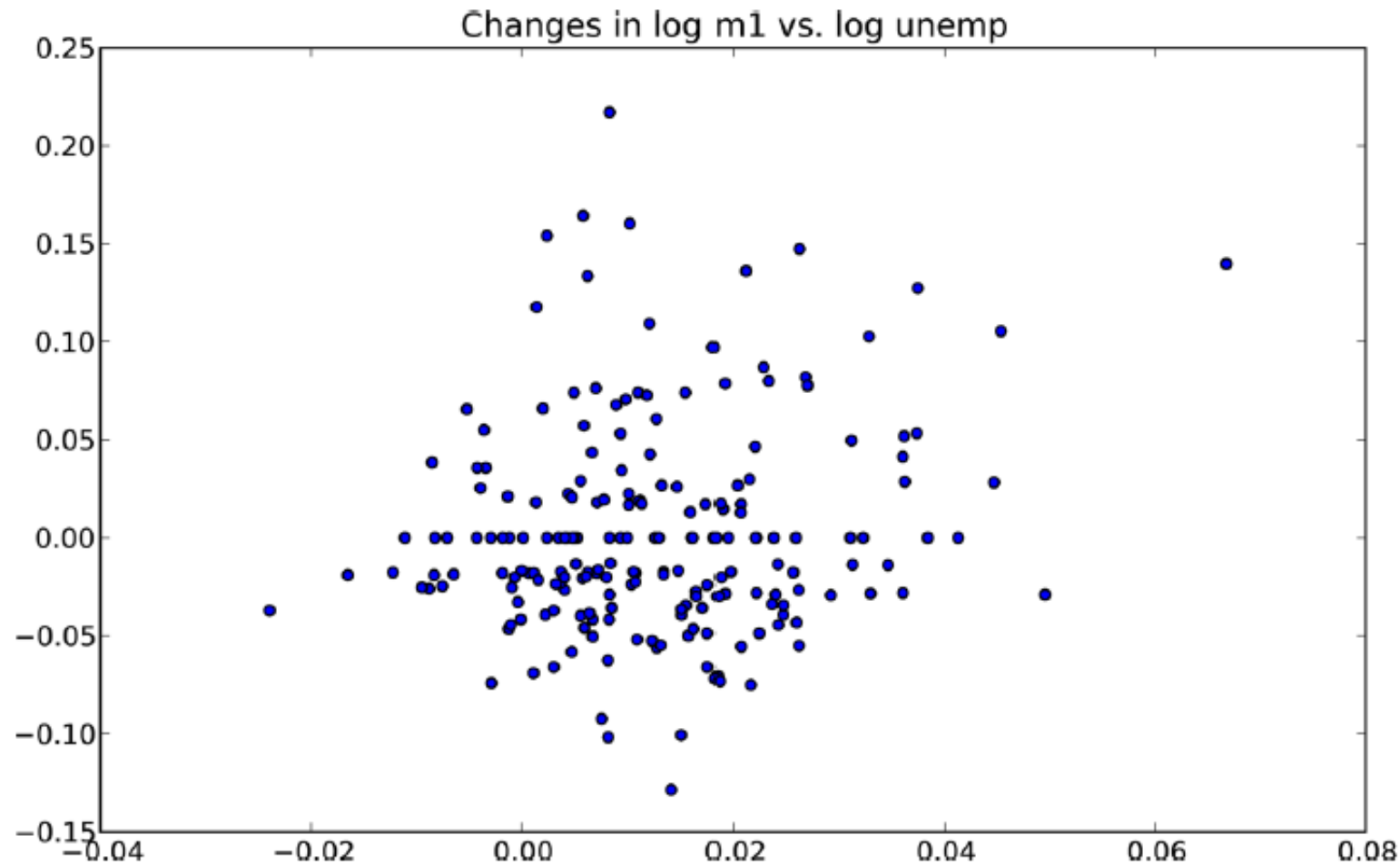


Figure 8-22. A simple scatter plot

Introduction to Web Scrapping and Ethical Consideration

Introduction to Web Scarping

Web Scraping:

- Web scraping is the process of automatically extracting information from websites.
- Instead of copying data manually, we use Python libraries (like requests, BeautifulSoup, or Scrapy) to fetch and parse HTML.

Web Scraping Works:

- Send a request to the website server.
- Get the HTML content of the page.
- Parse the HTML to find the required data.
- Save the data into a file (CSV, JSON, database).

Ethical Consideration

68

Ethical Considerations:

- Very important before scraping:
- Check the website's robots.txt (it defines scraping rules).
- Respect Terms of Service (ToS) of the site.
- Don't overload servers (use delays between requests).
- Use scraping for learning, research, or public data — not to steal content or spam.
- Never scrape personal or sensitive data (emails, passwords, credit card info).
- Don't try to bypass login/security systems.

Using Requests and BeautifulSoup Libraries

Requests Library:

Used to **send HTTP requests** (like visiting a website in a browser).

It returns the **HTML content** of a webpage.

```
import requests
```

```
url = "http://quotes.toscrape.com"
```

```
response = requests.get(url)
```

```
print("Status Code:", response.status_code) # 200 = success
```

```
print(response.text[:500]) # Print first 500 characters of HTML
```

Using Requests and BeautifulSoup Libraries

70

BeautifulSoup Library:

Used to parse HTML/XML and extract useful data.

Helps in navigating tags like <h1>, <p>, <div>, etc.

```
from bs4 import BeautifulSoup
# Convert HTML text to BeautifulSoup object
soup = BeautifulSoup(response.text, "html.parser")
# Find the first <h1> tag
heading = soup.find("h1")
print("Heading:", heading.text)
```

Using Requests and BeautifulSoup Libraries

71

Extracting Links:

We can grab all <a> (anchor) tags.

```
links = soup.find_all("a")  
for link in links[:5]: # show first 5 links  
    print(link.get("href"))
```

Using Requests and BeautifulSoup Libraries

72

Storing Scraped Data in a List:

```
data = []  
for i in range(len(quotes)):  
    data.append({  
        "quote": quotes[i].text,  
        "author": authors[i].text })  
print(data[:2]) # show first 2 items
```

Using Requests and BeautifulSoup Libraries

73

Saving Data to CSV:

```
import csv  
  
with open("quotes.csv", "w", newline="", encoding="utf-8") as file:  
    writer = csv.DictWriter(file, fieldnames=["quote", "author"])  
    writer.writeheader()  
    writer.writerows(data)
```

Extracting Data from Static Web Pages

74

- **Static Web Pages:**

A static web page is one where the content (HTML, CSS, images) is fixed and doesn't change unless the developer updates it.

Examples: Blogs, articles, documentation pages.

Unlike dynamic pages (which use JavaScript to load data on-the-fly), static pages send complete HTML directly to your browser.

Static pages are easier to scrape because the data is already in the HTML source.

Extracting Data from Static Web Pages

75

- **Steps to Extract Data:**

- Send a request to the webpage using requests.

- Parse the HTML using BeautifulSoup.

- Locate HTML tags containing the data (using tag name, class, or id).

- Extract the text or attributes (like links).

- Save data into CSV/JSON/database.

Introduction to APIs and JSON Format

76

- **API (Application Programming Interface):**

It is a way for two systems (apps, websites, software) to communicate with each other.

In web development, a Web API allows you to request data from a server.

Instead of scraping HTML, APIs usually provide structured data (often in JSON format).

- **JSON Format:**

JSON = JavaScript Object Notation

A lightweight format to store and transfer data.

Very popular in APIs because it's easy to read and language-independent.

JSON looks like Python dictionaries (key-value pairs).

Making API Request

77

We use the requests library to call an API and then parse its JSON response.

Using a Fake JSON API:

```
import requests

# Example API (returns JSON data)
url = "https://jsonplaceholder.typicode.com/users"
response = requests.get(url)

# Convert API response to JSON
data = response.json()

# Print first user
print(data[0])
```

Making API Request

78

Extract Specific Data from JSON:

```
for user in data[:5]:
```

```
    print("Name:", user["name"], " | Email:", user["email"])
```

Example

79

- Web Scrapping

```
from bs4 import BeautifulSoup
import requests
import pandas as pd
import html5lib as html
```

- Get the url

```
url = 'https://www.worldometers.info/coronavirus/'
req = requests.get(url)
req
```

Note : If output = "Response [200]". It work!! If output = "Response [404]. Page not found!!"

Example

80

- Parse html page

Data parsing is the process of transforming a sequence (unstructured data) into a tree or parse tree (structured data) that's easier to read, understand and use.

```
soup = BeautifulSoup(req.content, 'html.parser')
#print(soup) #To verify output (VERY LONG)
soup
```

```
# Use to find the table or other notations with required fields
table = soup.find("table", id="main_table_countries_today")
tbody = table.find("tbody")
```

```
for row in tbody.find_all("tr"):
    for td in row.find_all("td")[1:3]:
        print(td.text)
    print("-----")
```

Making API Requests using requests Module

81

- **API Request:**

An API request is how we communicate with a web API (Application Programming Interface).

It's just like your browser visiting a website, but instead of HTML, the server usually sends back structured data (JSON or XML).

We use Python's requests library to send these requests.

Making API Requests using requests Module

82

- **Steps for Making an API Request:**

Import the requests library.

Use `requests.get(url)` to send a GET request.

Check the status code (200 = success).

Parse the response as JSON using `.json()`.

Work with the data like a Python dictionary.

Making API Requests using requests Module

83

- **GET Request (Fake User Data API):**

We'll use a free test API: <https://jsonplaceholder.typicode.com/users>

```
import requests
```

```
# Step 1: API endpoint
```

```
url = "https://jsonplaceholder.typicode.com/users"
```

```
# Step 2: Send GET request
```

```
response = requests.get(url)
```

Making API Requests using requests Module

84

Step 3: Check status

```
print("Status Code:", response.status_code) # 200 means success
```

Step 4: Convert response to JSON

```
data = response.json()
```

Step 5: Print first user

```
print("First User:", data[0])
```

Making API Requests using requests Module

85

Extract Specific Data:

```
for user in data[:3]: # first 3 users  
    print("Name:", user["name"], "| Email:", user["email"])
```

Parsing and Using API Data

86

Parsing = breaking down raw API data (usually in JSON format) into usable parts.

APIs return JSON (key–value pairs, arrays), which in Python becomes a dictionary or list after calling `.json()`.

When parsed in Python, you can extract values like `data["name"]` or `data["address"]["city"]`.

Parsing and Using API Data

87

Steps to Parse API Data

Make the API request.

Convert the response into JSON (.json()).

Navigate through the dictionary/list.

Extract the fields you need.

Use or store the data (print, save to CSV/DB, make decisions).

Weather API

88

```
import requests

city = "Paris"

url = f"https://wttr.in/{city}?format=j1"

response = requests.get(url)

# Parse JSON

weather = response.json()

# Extract current condition

temp = weather["current_condition"][0]["temp_C"]

desc = weather["current_condition"][0]["weatherDesc"][0]["value"]

print(f"Weather in {city}: {temp}°C, {desc}")
```

Weather API

89

Using the Data:

After parsing, you can:

- Display it to users (like a weather app).

- Save it to CSV/JSON/database.

- Perform data analysis (e.g., average temperatures).

- Trigger actions (e.g., send an alert if weather = "Storm").

Authentication & Security Basics for APIs in Python⁹⁰

API Key Authentication:

A secret key (like a password) is given to you by the API provider.

You include it in the request header or URL.

```
import requests

url = "https://www.thesportsdb.com/api/v1/json/3/searchteams.php"

params = {"t": "Arsenal"} # team name

response = requests.get(url, params=params)

data = response.json()

print(data)
```

Authentication & Security Basics for APIs in Python⁹¹

Basic Authentication (Username + Password):

Send credentials with the request.

Mostly used in internal APIs, not very secure if not over HTTPS.

```
from requests.auth import HTTPBasicAuth
```

```
import requests
```

```
import base64
```

```
url = "https://postman-echo.com/basic-auth"
```

```
user, pwd = "postman", "password"
```

Authentication & Security Basics for APIs in Python⁹²

```
token = base64.b64encode(f"{user}:{pwd}".encode()).decode()
headers = {"Authorization": f"Basic {token}"}
response = requests.get(url, headers=headers)
print(response.status_code)
print(response.json())
```